

Executive Summary

The Aggie Clothing Store database intends to solve a problem where information about various Aggie Clothing retail outlets and their associated employees and inventories may be difficult to attain and derive insights from. The database contains information about these stores, including their locations, employees, managers, products, inventories, and sale histories. A high level manager would be able to query the database utilizing our provided example queries to gain detailed information about every store. This manager role would also be able to update the database to keep it relevant as the retail outlets change over time. All of this functionality will provide those in charge of the Aggie Clothing Store retail chain with vital information that informs their decisions for ordering inventory or managing employees.

Design Documents

The database life cycle encompasses all procedures done to a database spanning from the first idea of it to its long-term use. When examining the beginning of a database lifecycle, the first place to look is where a problem with a system arises. If there is a consistent mismanagement of information that plagues a company, team, or organization, the idea for a database to solve these issues may begin to arise. At this stage, user needs, logistics, and the size of the database are all considered as a plan is put into motion for implementation. This stage of the life cycle is important as it inherently justifies the existence of the database. If all initial design decisions are approached from a problem solving perspective, the finished product will be better suited to handle the issues that a team is having.

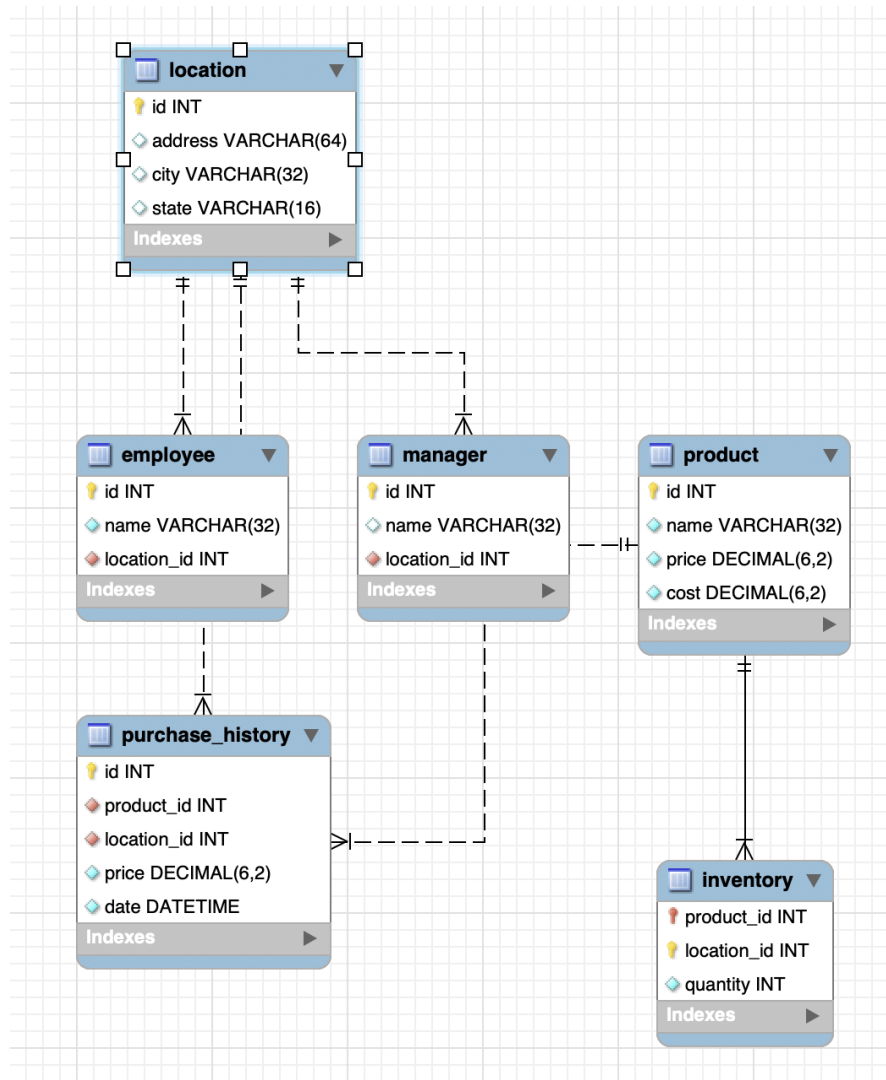
Following this stage, work will begin to be done in defining the relationship between tables and the data within them. Here, the ideas in the planning phase are acted upon as specific data types are used to capture real world information in a convenient and approachable format. Considerations for the earlier defined user requirements will also inform the structure of the database to ensure that queries can give users what they need during operation. This stage is vital to the life cycle as it defines the majority of the functionality for the database going forward.

After the design is implemented into software and thoroughly tested, the final stage involves monitoring the quality of the database overtime and responding to issues that arise. This can range from removing bugs that were not previously discovered to adding new functionality at the request of users. The needs of employees or team members can evolve over time, and flexibility is key in the life cycle to ensure that a database does not become

antiquated. This final phase of the life cycle ensures that user needs are continually being met and that support does not quickly drop off after the first draft of the database.

Including the documentation for the user requirements is essential to the design of a database as it gives the designers a clear path towards optimizing the user experience. With a solid foundation of what the user expects to be able to view, edit, and analyze, the database system's structure will be informed from the onset of development by the entire set of expectations. If every user requirement is not laid out before table and database design begins, large pieces of functionality may be left out throughout the process that cannot easily be rectified. Without full knowledge of what tables are needed, the schematic for the project is susceptible to failure when the hierarchy of tables suddenly changes when features are discovered to be missing.

Conceptual design is the process of creating a visual schematic that represents an idea for the final product of a project. It is the first phase of a successful project and is necessary to gather all of the ideas and features that should be carried through to the end goal. This design clearly lays out every entity, relationship, and piece of information that will be maintained in the database and gives everyone on the project and possibly investors a solid idea of what will be created. Our conceptual design is in the form of an Entity Relationship Diagram, which is a visual layout that shows every table, the data members of those tables, and the relationships between keys and data represented with arrows. An ERD is a blueprint that can be used to fully convey the expectations for a database in an approachable format. We have attached our ERD with plans for the Aggie Clothing Store database below:



Tables

- Manager
 - Id
 - Name
 - Location_id
 - (Primary key: id)
- Location (Manager manages this location, location only has one manager) (1-1 relation)
 - Id
 - Address
 - City
 - State
 - (Primary key: id)
- Employee (Location can have multiple employees, employee can only work at one location)

- Id
 - Name
 - Location_id
 - (Primary key: id)
- Inventory (Contains many products)
 - Product_id
 - Location_id
 - Quantity
 - (Composite key: product_id, location_id)
- Product
 - Id
 - Name
 - Price
 - Cost
 - (Primary key: id)
- Purchase_history
 - Id
 - Product_id
 - Location_id
 - Price
 - Date
 - (Primary key: id)

```
SET NAMES utf8mb4;
SET @@OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
SET @@OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS,
FOREIGN_KEY_CHECKS=0;
SET @@OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='TRADITIONAL';
```

```
DROP SCHEMA IF EXISTS aggie_clothing_store;
CREATE SCHEMA aggie_clothing_store;
USE aggie_clothing_store;
```

```
CREATE TABLE location (
  id INT NOT NULL,
  address VARCHAR(64),
  city VARCHAR(32),
  state VARCHAR(16),
  PRIMARY KEY (id)
);
```

```
CREATE TABLE product (
```

```

        id INT NOT NULL,
        name VARCHAR(32) NOT NULL,
        price DECIMAL(6, 2) NOT NULL,
        cost DECIMAL(6, 2) NOT NULL,
        PRIMARY KEY (id)
    );

CREATE TABLE manager (
    id INT NOT NULL,
    name VARCHAR(32),
    location_id INT NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY (location_id) REFERENCES location(id)
);

CREATE TABLE employee (
    id INT NOT NULL,
    name VARCHAR(32) NOT NULL,
    location_id INT NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY (location_id) REFERENCES location(id)
);

CREATE TABLE inventory (
    product_id INT NOT NULL,
    location_id INT NOT NULL,
    quantity INT NOT NULL,
    PRIMARY KEY (product_id, location_id),
    FOREIGN KEY (product_id) REFERENCES product(id)
);

CREATE TABLE purchase_history(
    id INT NOT NULL,
    product_id INT NOT NULL,
    location_id INT NOT NULL,
    price DECIMAL(6,2) NOT NULL,
    date DATETIME NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY (product_id) REFERENCES product(id),
    FOREIGN KEY (location_id) REFERENCES location(id)
);

```

Data Entry

```
INSERT INTO location (id, address, city, state)
VALUES
    (6, '123 Elm Street', 'Dallas', 'TX'),
    (7, '456 Oak Avenue', 'Denver', 'CO'),
    (8, '789 Pine Lane', 'Miami', 'FL'),
    (9, '101 Maple Drive', 'Chicago', 'IL'),
    (10, '202 Birch Blvd', 'Seattle', 'WA'),
    (11, '303 Cedar Road', 'Phoenix', 'AZ'),
    (12, '404 Spruce Street', 'Atlanta', 'GA'),
    (13, '505 Redwood Terrace', 'Boston', 'MA'),
    (14, '606 Willow Court', 'Detroit', 'MI'),
    (15, '707 Aspen Way', 'Las Vegas', 'NV'),
    (16, '808 Hickory Circle', 'Portland', 'OR'),
    (17, '909 Magnolia Plaza', 'Charlotte', 'NC'),
    (18, '1010 Juniper Square', 'Philadelphia', 'PA'),
    (19, '1111 Poplar Avenue', 'Nashville', 'TN'),
    (20, '1212 Cypress Street', 'San Diego', 'CA');
```

```
INSERT INTO product (id, name, price, cost)
VALUES
    (6, 'Gig 'Em Pants', 32.99, 15.96),
    (7, 'Good Luck Cap', 24.99, 14.89),
    (8, 'Coffee-Colored Mitts', 23.99, 7.56),
    (9, 'Horse-Print Shirt', 17.99, 7.52),
    (10, 'Low-Waist Patched Jeans', 95.99, 37.62),
    (11, 'A&M Hoodie', 45.99, 18.52),
    (12, 'AggieLand Shirt', 34.99, 12.90),
    (13, 'Aggie Football Jersey', 52.99, 24.89),
    (14, 'Reveille Mug', 20.00, 5.72),
    (15, 'AggieTexas Keychain', 7.99, 2.50),
    (16, 'AggieStar Shoes', 105.00, 45.95),
    (17, 'Reveille Fuzzy Socks', 12.99, 3.93),
    (18, 'Mess with Texas T-Shirt', 34.99, 17.25),
    (19, 'Saw Em Off Game T-Shirt', 32.99, 15.00),
    (20, 'BTHO of Texas T-Shirt', 32.99, 15.50);
```

```
INSERT INTO manager (id, name, location_id)
VALUES
    (1, 'George Phillips', 1),
    (2, 'Barbara Wright', 2),
    (3, 'Charles Young', 3),
    (4, 'Edward Baker', 4),
    (5, 'Diana Green', 5),
```

```
(6, 'Ian Kelly', 6),
(7, 'Andrew King', 7),
(8, 'Hannah Mitchell', 8),
(9, 'Fiona Carter', 9),
(10, 'Jessica Adams', 10),
(11, 'Michael Parker', 11),
(12, 'Laura Turner', 12),
(13, 'Kevin Collins', 13),
(14, 'Oliver Jenkins', 14),
(15, 'Natalie Stewart', 15);
```

```
INSERT INTO employee (id, name, location_id)
VALUES
```

```
(1, 'Alice Johnson', 1),
(2, 'Eve Wilson', 1),
(3, 'Carol Davis', 1),
(4, 'David Brown', 2),
(5, 'Bob Smith', 2),
(6, 'Frank Garcia', 3),
(7, 'Grace Martinez', 3),
(8, 'Leo Hernandez', 3),
(9, 'Ivy Taylor', 4),
(10, 'Jack Anderson', 4),
(11, 'Karen Thomas', 5),
(12, 'Hank Moore', 5),
(13, 'Mia Martin', 5),
(14, 'Paula Hall', 6),
(15, 'Oscar Clark', 6),
(16, 'Nina LopezPaula Hall', 6),
(17, 'Quincy Allen', 7),
(18, 'Rita Adams', 7),
(19, 'Steve Baker', 8),
(20, 'Tina Scott', 8),
(21, 'Timothy Gonzales', 9),
(22, 'Victor Hill', 9),
(23, 'Wendy Cooper', 10),
(24, 'Xander Flores', 10),
(25, 'Yara Rogers', 10),
(26, 'Zane Parker', 11),
(27, 'Aaron Sanders', 12),
(28, 'Beth Carter', 12),
(29, 'Chris Evans', 12),
(30, 'Diane Powell', 13),
(31, 'Ethan Stewart', 13),
(32, 'Faith Ward', 14),
(33, 'George Hughes', 14),
(34, 'Holly Bennett', 15),
```

```
(35, 'Ian Foster', 15);
```

```
INSERT INTO inventory (product_id, location_id, quantity)
VALUES
```

```
-- Location 1
```

```
(1, 1, 10), (2, 1, 18), (3, 1, 7), (4, 1, 12), (5, 1, 0), (6, 1, 9),
(7, 1, 34), (8, 1, 11), (9, 1, 19), (10, 1, 13), (11, 1, 0), (12, 1, 16),
(13, 1, 50), (14, 1, 12), (15, 1, 7), (16, 1, 17), (17, 1, 55),
(18, 1, 6), (19, 1, 0), (20, 1, 0),
```

```
-- Location 2
```

```
(1, 2, 14), (2, 2, 6), (3, 2, 0), (4, 2, 9), (5, 2, 49), (6, 2, 8),
(7, 2, 7), (8, 2, 18), (9, 2, 15), (10, 2, 0), (11, 2, 13), (12, 2, 5),
(13, 2, 87), (14, 2, 10), (15, 2, 11), (16, 2, 0), (17, 2, 14),
(18, 2, 8), (19, 2, 48), (20, 2, 7),
```

```
-- Location 3
```

```
(1, 3, 8), (2, 3, 1), (3, 3, 14), (4, 3, 6), (5, 3, 72), (6, 3, 11),
(7, 3, 18), (8, 3, 9), (9, 3, 0), (10, 3, 20), (11, 3, 7), (12, 3, 17),
(13, 3, 10), (14, 3, 0), (15, 3, 15), (16, 3, 89), (17, 3, 6),
(18, 3, 14), (19, 3, 9), (20, 3, 16),
```

```
-- Location 4
```

```
(1, 4, 13), (2, 4, 15), (3, 4, 19), (4, 4, 0), (5, 4, 11),
(6, 4, 0), (7, 4, 95), (8, 4, 14), (9, 4, 58), (10, 4, 7),
(11, 4, 12), (12, 4, 67), (13, 4, 23), (14, 4, 6), (15, 4, 43),
(16, 4, 18), (17, 4, 0), (18, 4, 9), (19, 4, 20), (20, 4, 11),
```

```
-- Location 5
```

```
(1, 5, 14), (2, 5, 17), (3, 5, 10), (4, 5, 20), (5, 5, 9),
(6, 5, 9), (7, 5, 0), (8, 5, 12), (9, 5, 8), (10, 5, 0),
(11, 5, 15), (12, 5, 10), (13, 5, 16), (14, 5, 3), (15, 5, 0),
(16, 5, 13), (17, 5, 19), (18, 5, 0), (19, 5, 14), (20, 5, 11),
```

```
-- Location 6
```

```
(1, 6, 12), (2, 6, 15), (3, 6, 0), (4, 6, 18), (5, 6, 8),
(6, 6, 0), (7, 6, 13), (8, 6, 6), (9, 6, 11), (10, 6, 14),
(11, 6, 9), (12, 6, 0), (13, 6, 7), (14, 6, 0), (15, 6, 0),
(16, 6, 10), (17, 6, 8), (18, 6, 15), (19, 6, 17), (20, 6, 9),
```

```
-- Location 7
```

```
(1, 7, 6), (2, 7, 13), (3, 7, 10), (4, 7, 0), (5, 7, 15),
(6, 7, 12), (7, 7, 0), (8, 7, 8), (9, 7, 7), (10, 7, 11),
(11, 7, 9), (12, 7, 16), (13, 7, 0), (14, 7, 10), (15, 7, 14),
(16, 7, 18), (17, 7, 19), (18, 7, 6), (19, 7, 0), (20, 7, 8),
```

```

-- Location 8
(1, 8, 20), (2, 8, 18), (3, 8, 16), (4, 8, 0), (5, 8, 13),
(6, 8, 0), (7, 8, 0), (8, 8, 7), (9, 8, 14), (10, 8, 0),
(11, 8, 9), (12, 8, 15), (13, 8, 12), (14, 8, 19), (15, 8, 0),
(16, 8, 10), (17, 8, 0), (18, 8, 13), (19, 8, 14), (20, 8, 17),

-- Location 9
(1, 9, 11), (2, 9, 9), (3, 9, 7), (4, 9, 19), (5, 9, 6),
(6, 9, 38), (7, 9, 8), (8, 9, 14), (9, 9, 3), (10, 9, 10),
(11, 9, 12), (12, 9, 0), (13, 9, 16), (14, 9, 2), (15, 9, 82),
(16, 9, 18), (17, 9, 74), (18, 9, 15), (19, 9, 11), (20, 9, 9),

-- Location 10
(1, 10, 8), (2, 10, 12), (3, 10, 15), (4, 10, 10), (5, 10, 0),
(6, 10, 81), (7, 10, 0), (8, 10, 11), (9, 10, 7), (10, 10, 14),
(11, 10, 9), (12, 10, 13), (13, 10, 7), (14, 10, 18), (15, 10, 6),
(16, 10, 10), (17, 10, 17), (18, 10, 7), (19, 10, 20), (20, 10, 12),

-- Location 11
(1, 11, 14), (2, 11, 9), (3, 11, 0), (4, 11, 17), (5, 11, 6),
(6, 11, 10), (7, 11, 13), (8, 11, 0), (9, 11, 15), (10, 11, 19),
(11, 11, 8), (12, 11, 0), (13, 11, 11), (14, 11, 16), (15, 11, 7),
(16, 11, 0), (17, 11, 12), (18, 11, 18), (19, 11, 9), (20, 11, 5),

-- Location 12
(1, 12, 0), (2, 12, 14), (3, 12, 12), (4, 12, 6), (5, 12, 19),
(6, 12, 11), (7, 12, 0), (8, 12, 8), (9, 12, 10), (10, 12, 0),
(11, 12, 16), (12, 12, 7), (13, 12, 15), (14, 12, 9), (15, 12, 18),
(16, 12, 0), (17, 12, 13), (18, 12, 20), (19, 12, 6), (20, 12, 8),

-- Location 13
(1, 13, 18), (2, 13, 10), (3, 13, 7), (4, 13, 14), (5, 13, 0),
(6, 13, 9), (7, 13, 16), (8, 13, 0), (9, 13, 0), (10, 13, 8),
(11, 13, 19), (12, 13, 6), (13, 13, 0), (14, 13, 12), (15, 13, 15),
(16, 13, 10), (17, 13, 17), (18, 13, 13), (19, 13, 0), (20, 13, 7),

-- Location 14
(1, 14, 9), (2, 14, 8), (3, 14, 11), (4, 14, 16), (5, 14, 10),
(6, 14, 0), (7, 14, 12), (8, 14, 0), (9, 14, 14), (10, 14, 20),
(11, 14, 7), (12, 14, 13), (13, 14, 0), (14, 14, 15), (15, 14, 0),
(16, 14, 6), (17, 14, 19), (18, 14, 36), (19, 14, 17), (20, 14, 9),

-- Location 15
(1, 15, 8), (2, 15, 19), (3, 15, 15), (4, 15, 0), (5, 15, 12),
(6, 15, 9), (7, 15, 11), (8, 15, 16), (9, 15, 0), (10, 15, 14),
(11, 15, 6), (12, 15, 0), (13, 15, 17), (14, 15, 0), (15, 15, 18),
(16, 15, 24), (17, 15, 0), (18, 15, 7), (19, 15, 20), (20, 15, 5);

```

```

INSERT INTO product (id, name, price, cost)
VALUES
    (6, 'Gig 'Em Pants', 32.99, 15.96),
    (7, 'Good Luck Cap', 24.99, 14.89),
    (8, 'Coffee-Colored Mitts', 23.99, 7.56),
    (9, 'Horse-Print Shirt', 17.99, 7.52),
    (10, 'Low-Waist Patched Jeans', 95.99, 37.62),
    (11, 'A&M Hoodie', 45.99, 18.52),
    (12, 'AggieLand Shirt', 34.99, 12.90),
    (13, 'Aggie Football Jersey', 52.99, 24.89),
    (14, 'Reveille Mug', 20.00, 5.72),
    (15, 'AggieTexas Keychain', 7.99, 2.50),
    (16, 'AggieStar Shoes', 105.00, 45.95),
    (17, 'Reveille Fuzzy Socks', 12.99, 3.93),
    (18, 'Mess with Texas T-Shirt', 34.99, 17.25),
    (19, 'Saw Em Off Game T-Shirt', 32.99, 15.00),
    (20, 'BTHO of Texas T-Shirt', 32.99, 15.50);

```

```

INSERT INTO purchase_history (id, product_id, location_id, price, date)
VALUES
    (1, 3, 7, 19.50, '2024-10-10 14:23:45'),
    (2, 15, 4, 45.99, '2024-09-15 09:15:30'),
    (3, 9, 12, 62.55, '2024-08-22 16:45:00'),
    (4, 6, 3, 52.99, '2024-07-18 11:05:22'),
    (5, 11, 8, 45.00, '2024-10-05 13:22:10'),
    (6, 2, 15, 32.99, '2024-06-12 10:30:50'),
    (7, 10, 5, 24.99, '2024-11-01 17:40:08'),
    (8, 1, 9, 23.99, '2024-10-27 20:50:35'),
    (9, 8, 14, 17.99, '2024-04-03 18:11:01'),
    (10, 7, 11, 95.99, '2024-05-22 14:09:58'),
    (11, 4, 6, 45.99, '2024-07-30 07:25:17'),
    (12, 5, 2, 34.99, '2024-09-19 19:55:42'),
    (13, 20, 10, 52.99, '2024-11-10 08:21:30'),
    (14, 12, 13, 20.00, '2024-03-15 11:10:05'),
    (15, 14, 1, 7.99, '2024-02-11 22:35:10'),
    (16, 16, 7, 105.00, '2024-08-04 16:18:47'),
    (17, 18, 4, 12.99, '2024-06-25 23:05:12'),
    (18, 19, 9, 34.99, '2024-10-19 14:40:11'),
    (19, 13, 5, 32.99, '2024-09-07 10:14:57'),
    (20, 17, 3, 32.99, '2024-01-21 06:00:34'),
    (21, 6, 1, 19.50, '2024-04-10 13:50:25'),
    (22, 3, 15, 45.99, '2024-10-11 09:30:17'),
    (23, 7, 14, 62.55, '2024-02-14 08:00:02'),
    (24, 11, 12, 52.99, '2024-03-22 12:25:48'),
    (25, 1, 8, 45.00, '2024-09-28 21:13:55'),

```

(26, 4, 9, 32.99, '2024-05-19 15:32:04'),
(27, 18, 13, 24.99, '2024-06-21 17:45:39'),
(28, 15, 2, 23.99, '2024-07-10 11:20:47'),
(29, 9, 6, 17.99, '2024-08-15 14:55:30'),
(30, 20, 7, 95.99, '2024-10-01 20:00:58'),
(31, 10, 11, 45.99, '2024-01-13 08:30:05'),
(32, 5, 4, 34.99, '2024-09-07 06:45:23'),
(33, 12, 5, 52.99, '2024-07-02 19:11:04'),
(34, 14, 14, 20.00, '2024-03-29 07:28:13'),
(35, 19, 10, 7.99, '2024-06-06 16:19:37'),
(36, 13, 8, 105.00, '2024-05-30 10:41:05'),
(37, 2, 9, 12.99, '2024-02-20 13:55:26'),
(38, 8, 6, 34.99, '2024-09-18 12:12:33'),
(39, 16, 3, 32.99, '2024-10-15 15:35:10'),
(40, 17, 15, 32.99, '2024-11-02 07:20:48'),
(41, 1, 2, 19.50, '2024-04-05 11:25:57'),
(42, 3, 11, 45.99, '2024-07-09 21:50:38'),
(43, 7, 13, 62.55, '2024-02-14 14:05:11'),
(44, 11, 7, 52.99, '2024-02-28 16:00:22'),
(45, 6, 12, 45.00, '2024-09-12 10:15:46'),
(46, 18, 4, 32.99, '2024-06-16 08:05:13'),
(47, 20, 1, 23.99, '2024-10-25 19:13:35'),
(48, 14, 14, 17.99, '2024-01-11 13:55:22'),
(49, 12, 9, 34.99, '2024-08-22 22:20:40'),
(50, 9, 5, 32.99, '2024-09-16 09:40:14'),
(51, 4, 15, 32.99, '2024-05-01 17:10:03'),
(52, 8, 6, 24.99, '2024-02-25 18:30:47'),
(53, 10, 3, 23.99, '2024-09-09 14:25:31'),
(54, 17, 11, 17.99, '2024-06-08 16:55:52'),
(55, 15, 7, 95.99, '2024-01-15 08:15:05'),
(56, 19, 8, 45.99, '2024-03-14 19:25:41'),
(57, 2, 10, 52.99, '2024-10-23 10:50:24'),
(58, 13, 2, 20.00, '2024-04-14 13:13:42'),
(59, 5, 12, 7.99, '2024-07-17 14:22:14'),
(60, 16, 13, 105.00, '2024-06-19 20:55:39'),
(61, 1, 14, 12.99, '2024-08-30 22:40:28'),
(62, 20, 4, 34.99, '2024-02-22 11:55:18'),
(63, 3, 15, 32.99, '2024-07-15 08:35:11'),
(64, 7, 8, 24.99, '2024-03-26 18:05:00'),
(65, 9, 9, 17.99, '2024-05-13 16:45:56'),
(66, 14, 11, 52.99, '2024-09-25 11:15:03'),
(67, 18, 5, 45.00, '2024-06-01 09:30:20'),
(68, 6, 10, 32.99, '2024-01-07 12:00:00'),
(69, 8, 6, 19.50, '2024-04-18 10:35:30'),
(70, 2, 7, 45.99, '2024-08-28 16:23:40'),
(71, 13, 3, 52.99, '2024-02-19 14:05:22'),
(72, 17, 9, 7.99, '2024-10-08 11:15:02'),

```
(73, 19, 13, 34.99, '2024-09-11 13:55:45'),  
(74, 4, 14, 32.99, '2024-07-14 22:30:59'),  
(75, 12, 8, 32.99, '2024-10-19 08:25:18');
```

Critical Analysis

Our final database design matched our original plans very closely, varying only in a few small ways related to the number of columns in each table. The decisions we made when changing the database were informed mostly by the desire to cut down on the excessive “join” statements that would have been needed to read information from the database properly. For example, our purchase history table initially had significantly fewer columns and was made under the assumption that “join” statements would fill in some of the gaps for sales records. However, we found that the exclusion of a total price listed for each sale record added a significant amount of processing time when trying to determine how much money changed hands in a given transaction. Other than the addition of columns such as these, our original design allowed us to act on our planned queries as expected and navigate the data to see the relationships between employees, managers, and locations with direct and streamlined queries.

Forms

Address

Insert Address

City

Insert City

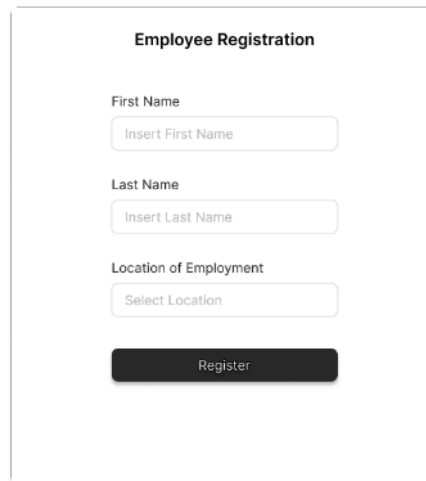
State

Insert State

Submit New Location

Locations

Id	Address	City	State
1	123 Elm Street	Dallas	TX
2	456 Oak Avenue	Denver	CO
3	789 Pine Lane	Miami	FL
4	101 Maple Drive	Chicago	IL
5	202 Birch Blvd	Seattle	WA
6	303 Cedar Road	Phoenix	AZ
7	404 Spruce Street	Atlanta	GA
8	505 Redwood Terran	Boston	MA
9	606 Willow Court	Detroit	MI
10	707 Aspen Way	Las Vegas	NV
11	808 Hickory Circle	Portland	OR

A screenshot of a web form titled "Employee Registration". The form is centered on a white background. It contains three input fields: "First Name" with a placeholder "Insert First Name", "Last Name" with a placeholder "Insert Last Name", and "Location of Employment" with a placeholder "Select Location". Below these fields is a dark grey button labeled "Register".

Employee Registration

First Name
Insert First Name

Last Name
Insert Last Name

Location of Employment
Select Location

Register

The main form is a concise table listing all locations on the database, as locations are the ubiquitous unit of the problem at hand. In addition to users being able to read information about all locations, there are input fields allowing users to easily update the database with new locations. This could be accessible by high-level managers wishing to register a newly erected location. The related form is a registration portal for new employees, allowing them to easily update the table with their information without allowing access to potentially sensitive information regarding other employees.

If we anticipated usage from an older adult population, it would be good to account for any potentially usability refinements. Many older adults have vision impairments, so clear fonts in bold style with contrastive colors would increase ease of access. Additionally, Some older adults with less computer experience may be overwhelmed by large amounts of content on screen at once. This can be avoided by keeping design uncluttered and concise, while still conferring enough information to be intuitive.

Queries

- 1.) Product name, price, and cost joined on inventory table (JOIN); shows more information about the products on hand to the users
- 2.) Total sales of a specific day for a specific location (AGGREGATE); used to analyze the productivity of a store and possibly compare to other stores
- 3.) Most popular item for the week (sidenote: all locations) (GROUP); used to detect what is popular among customers and possibly increase inventory
- 4.) Sort products by price (SORT); shows what is offered across the retail chain from most to least expensive

- 5.) All employees from a location; shows everyone that works at a given store
- 6.) All employees a manager manages (JOIN); shows every employee managed by a particular manager, can be used by a manager to keep track of their staff

1.) SELECT product.name, product.price, product.cost FROM inventory INNER JOIN product ON inventory.product_id=product.id;

	name	price	cost	
	flamingo shirt	19.50	5.73	
	flamingo shirt	19.50	5.73	
	flamingo shirt	19.50	5.73	
	flamingo shirt	19.50	5.73	
	flamingo shirt	19.50	5.73	
	flamingo shirt	19.50	5.73	
	flamingo shirt	19.50	5.73	
	flamingo shirt	19.50	5.73	
	flamingo shirt	19.50	5.73	
	flamingo shirt	19.50	5.73	
	flamingo shirt	19.50	5.73	
	flamingo shirt	19.50	5.73	
	flamingo shirt	19.50	5.73	
	flamingo shirt	19.50	5.73	
	blue jeans	45.99	18.50	
	blue jeans	45.99	18.50	
	blue jeans	45.99	18.50	
	blue jeans	45.99	18.50	
	blue jeans	45.99	18.50	
	blue jeans	45.99	18.50	
	blue jeans	45.99	18.50	
	blue jeans	45.99	18.50	
	blue jeans	45.99	18.50	
	blue jeans	45.99	18.50	
	blue jeans	45.99	18.50	
	blue jeans	45.99	18.50	
	blue jeans	45.99	18.50	
	blue jeans	45.99	18.50	
	blue jeans	45.99	18.50	
	black-laced...	62.55	25.56	
	black-laced...	62.55	25.56	
	black-laced...	62.55	25.56	

2.) SELECT SUM(price) FROM purchase_history WHERE location_id = 3;

SUM(price)
195.95

3.) SELECT product_id, COUNT(*) AS product_count FROM purchase_history WHERE date BETWEEN '2024-10-01 00:00:00' AND '2024-01-07 23:59:59' GROUP BY product_id ORDER BY product_count DESC LIMIT 1;

product_id	product_count
4	4

4.) SELECT * FROM product ORDER BY price DESC;

id	name	price	cost
16	AggieStar Shoes	105.00	45.95
10	Low-Waist Patched Jeans	95.99	37.62
3	black-laced shoes	62.55	25.56
4	striped puffer jacket	52.99	30.78
13	Aggie Football Jersey	52.99	24.89
2	blue jeans	45.99	18.50
11	A&M Hoodie	45.99	18.52
5	grey jacket	45.00	20.00
12	AggieLand Shirt	34.99	12.90
18	Mess with Texas T-Shirt	34.99	17.25
6	Gig 'Em Pants	32.99	15.96
19	Saw Em Off Game T-Shirt	32.99	15.00
20	BTHO of Texas T-Shirt	32.99	15.50
7	Good Luck Cap	24.99	14.89
8	Coffee-Colored Mitts	23.99	7.56
14	Reveille Mug	20.00	5.72
1	flamingo shirt	19.50	5.73
9	Horse-Print Shirt	17.99	7.52
17	Reveille Fuzzy Socks	12.99	3.93
15	AggieTexas Keychain	7.99	2.50
NULL	NULL	NULL	NULL

5.) SELECT * FROM employee WHERE location_id = 2;

id	name	location_id
4	David Brown	2
5	Bob Smith	2
NULL	NULL	NULL

6.) SELECT employee.name FROM employee INNER JOIN manager ON
employee.location_id = manager.location_id WHERE manager.id = 5;

name
Karen Thomas
Hank Moore
Mia Martin